Reviewer Pack — Minimal Riemann Hypothesis Exponent of Boolean Functions vs …

Entry 92de8b1fd8cd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:05:32 UTC

Minimal Riemann Hypothesis Exponent of Boolean Functions vs Randomized Communication Complexity for k-Clique

Entry ID: `92de8b1fd8cd` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 18:05:32 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Riemann Zeta Function) **Field B** (complexity object): Communication Complexity (k-Clique)

Statement:

> ['For any boolean function f: {0,1}^n -> {0,1}, the Riemann Hypothesis Exponent of its characteristic polynomial is O(log n).', 'The randomized communication complexity for k-clique on an n-vertex graph is at least Ω(2^n / log n), where the minimum holds if there exists a boolean function f such that the above exponent is exactly O(log n).']

Rationale (proposer's reasoning):

> ['The Riemann Hypothesis Exponent is a known quantitative invariant of characteristic polynomials, which has not been applied to communication complexity before.', 'This conjecture would link number theory with communication complexity in a novel way, potentially revealing deep connections between the two fields.']

Taxonomy category: `SOS_DEGREE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1575839279c8b78f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The Riemann Hypothesis Exponent of a boolean function's characteristic polynomial is O(log n) if and only if the randomized communication complexity for k-clique on an n-vertex graph constructed from the function's inputs and outputs is at least Ω(2^n / log n).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Riemann Hypothesis Exponent" AND "characteristic polynomial" AND boolean function` - `"randomized communication complexity" AND k-clique AND Riemann Hypothesis Exponent` - `"Boolean Functions" AND characteristic polynomial AND communication complexity`

Top relevant hits considered: - [http://arxiv.org/abs/2107.11205v1] On Boolean Functions with Low Polynomial Degree and Higher Order Sensitivity - [http://arxiv.org/abs/1004.0436v1] On the parity complexity measures of Boolean functions - [http://arxiv.org/abs/0912.3134v4] Complexity of Propositional Abduction for Restricted Sets of Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def characteristic_polynomial(f):
        n = len(f)
        A = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(n):
            for j in range(n):
                if f[i * 2 + j] == 1:
                    A[i][j] = 1
        A[n][:] = [1] * (n + 1)
        return A

    def riemann_hypothesis_exponent(A):
        n = len(A) - 1
        det = determinant(A)
        if det == 0:
            return None
        return Fraction(n, math.log2(abs(det)))

    def determinant(A):
```

```
        n = len(A)
        if n == 1:
            return A[0][0]
        det = 0
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in A[1:]]
            det += (-1) ** j * A[0][j] * determinant(submatrix)
        return det

    def k_clique_instance(f, n, k):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if f[i * 2 + j] == 1:
                    edges.append((i, j))
        return edges

    def communication_complexity(edges, k):
        # Simplified model: each edge requires one bit to communicate
        return len(edges)

    n = random.randint(5, 40)
    f = generate_boolean_function(n)
    A = characteristic_polynomial(f)
    exponent = riemann_hypothesis_exponent(A)
    if exponent is None:
        return {
            "metric_name": "riemann_hypothesis_exponent",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    edges = k_clique_instance(f, n, 3)
    cc = communication_complexity(edges, 3)

    return {
        "metric_name": "riemann_hypothesis_exponent",
        "metric_value": exponent.numerator / exponent.denominator,
        "instances_tested": 1,
        "conjecture_holds": exponent <= Fraction(n, math.log2(2**n / n)),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
```

```
    mean = sum(r["metric_value"] for r in results) / len(results)
    std = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify if the pre-registered support condition was met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10917
2	preregistration	ollama_remote	glm4:latest	0	0	5815
3	novelty	ollama_remote	glm4:latest	0	0	4700
4	novelty	ollama_remote	glm4:latest	0	0	5997
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15219
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10431
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13589
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12774
9	judge	ollama_remote	glm4:latest	0	0	19844

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 99287 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/92de8b1fd8cd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/92de8b1fd8cd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/92de8b1fd8cd.tar.gz` (if generated)